

Developer Documentation: Timetabling System

Maintainers of the jh03-main repository

February 4, 2026

Contents

1	Overview	2
1.1	What this system does	2
1.2	High-level goals for maintainers	2
1.3	How to navigate this document	2
2	Architecture	2
2.1	Repository layout	2
2.2	Runtime boundaries	2
2.3	Key data flows (summary)	3
3	Backend overview (Django)	3
3.1	Project entry points	3
3.2	Apps and responsibilities	3
3.3	Settings and environment	3
3.4	Permissions and roles	3
3.5	Next sections	4
4	Ingestion pipeline overview	4
4.1	Upload sequence	4
4.2	Key modules	4
4.3	Matching logic (high-level)	4
4.4	Error handling and data hygiene	4
5	Frontend overview (React)	5
5.1	Entry points and configuration	5
5.2	Routes and role gates	5
5.3	API access and auth storage	5
5.4	UI structure	5
6	Operations and CI	5
6.1	Docker Compose dev stack	5
6.2	Makefile targets	6
6.3	Refresh script	6
6.4	Continuous integration	6
6.5	Environments and secrets	6

7	Domain model reference	6
7.1	Enum and choice types	6
7.2	Core domain tables	7
7.3	Notifications and announcements	7
7.4	Invigilator management	7
7.5	Where to look next	7
8	API overview	7
8.1	URL routing	8
8.2	Authentication endpoints	8
8.3	Admin CRUD endpoints	8
8.4	Ingestion and exports	8
8.5	Notifications and messaging	8
8.6	Invigilator self-service endpoints	8
8.7	Serializers	9
8.8	Permissions and throttling	9
9	Services and utilities	9
9.1	Service layer	9
9.2	Ingestion helpers (upload_processor.py)	9
9.3	Venue matching (venue_matching.py)	9
9.4	Utilities	10
9.5	Cross-references	10
10	Testing strategy	10
10.1	Backend tests (Django)	10
10.2	Frontend tests (Vitest)	10
10.3	Running tests	11
10.4	Coverage	11
11	Accounts and authentication	11
11.1	Models	11
11.2	Authentication flow	11
11.3	Session management endpoints	11
11.4	Login restrictions	11
11.5	Admin integration	11
11.6	Password policy	12
12	Signals and implicit behaviors	12
12.1	Signal registration	12
12.2	Venue signals	12
12.3	Model-level hooks	12
13	Frontend component map	12
13.1	Routing layout	12
13.2	Admin pages	12
13.3	Invigilator pages	13
13.4	Admin dialogs and shared components	13
13.5	Cross-references	13

14 End-to-end data flow	13
14.1 Upload endpoint	13
14.2 File classification and parsing	13
14.3 Ingestion and persistence	14
14.4 Allocation and matching	14
14.5 API serialization to UI	14
15 Glossary and module map	14
15.1 Glossary	14
15.2 Module map	15
15.3 Known gaps	15
16 Upload processor reference	15
16.1 Public endpoint	15
16.2 Parsing and normalization helpers	15
16.3 Exam ingestion	16
16.4 Provision ingestion	16
16.5 Venue availability ingestion	16
16.6 Cross-references	17
17 API endpoint table	17
17.1 Authentication	17
17.2 Admin CRUD viewsets	17
17.3 Uploads and exports	17
17.4 Notifications and messaging	18
17.5 Invigilator self-service	18
17.6 Student/provision management	18
17.7 Cross-references	19
18 API views reference	19
18.1 Helper functions	19
18.2 Permission helpers	19
18.3 Export and upload views	19
18.4 CRUD viewsets	19
18.5 Notifications and messaging	20
18.6 Invigilator self-service	20
18.7 Student/provision management	20
18.8 Notes	20
19 API serializer reference	20
19.1 Exam and venue serializers	20
19.2 Notification and announcement serializers	21
19.3 Invigilator serializers	21
19.4 Diet serializer	21
19.5 Cross-references	21
20 Model detail reference	21
20.1 Choice types	21
20.2 Core tables	22

20.3	Notifications and uploads	22
20.4	Invigilator management	22
20.5	Behavioral hooks	22
21	Frontend component reference (detailed)	22
21.1	Pages	23
21.2	Shared components	23
21.3	Admin components	23
21.4	Invigilator components	24
21.5	Cross-references	24
22	Ops scripts and container images	24
22.1	Docker Compose (dev)	24
22.2	Django Dockerfile	24
22.3	Frontend Dockerfile	24
22.4	Django runtime scripts	25
22.5	Frontend runtime scripts	25
22.6	Cross-references	25
23	Accounts app reference	25
23.1	Models	25
23.2	Auth API helpers and views	25
23.3	Authentication backend	26
23.4	Adapters and validators	26
23.5	Admin integration	26
24	Backend utilities reference	26
24.1	Venue matching and stats	26
24.2	Excel parsing and classification	26
24.3	Column normalization	27
24.4	Venue parsing and ingest	27
24.5	Cross-references	27
25	Frontend state and data flow	27
25.1	Auth state	27
25.2	Admin data flow (common patterns)	28
25.3	Invigilator data flow	28
25.4	Cross-references	28
26	Accounts tests and migrations	28
26.1	Default admin migration	28
26.2	Core account tests	28
26.3	Extended auth tests	28
26.4	Password validator tests	29
27	Backend test index	29
28	Frontend tests and config	29
28.1	Vitest configuration	29

28.2 Frontend test index	30
29 Django views, URLs, and throttling	30
29.1 timetabling_system views	30
29.2 pages app views	30
29.3 URL routing	31
29.4 Throttling	31
29.5 Cross-references	31
30 Frontend utilities and state	31
30.1 Utilities	31
30.2 State management	31
30.3 Cross-references	31

1 Overview

This document is a developer-focused guide to the jh03-main codebase. It explains how the system is structured, how data flows, and where to make changes safely. The document is organized into small, linkable sections with references between related components.

1.1 What this system does

The application ingests large Excel exports that describe exam schedules, venues, and student provisions. It normalizes and stores those records in a database, then provides administrative and invigilator views for managing assignments, availability, and notifications. The frontend is a React application; the backend is a Django API.

1.2 High-level goals for maintainers

- Keep ingestion rules stable; it is the core business logic.
- Preserve referential integrity between exams, venues, and students.
- Ensure admin and invigilator permissions are enforced consistently.
- Keep Docker-driven local dev and CI flows reliable.

1.3 How to navigate this document

- Architecture and data flow: see [Section 2](#).
- Django backend details: see [Section 3](#).
- Ingestion pipeline details: see [Section 4](#).
- Frontend details: see [Section 5](#).
- Operations and CI: see [Section 6](#).

2 Architecture

The system is a two-service stack: a Django backend and a React frontend. Both are run locally via Docker Compose and in CI with container builds. The backend owns data ingestion, persistence, and API endpoints. The frontend owns UI workflows for admins and invigilators.

2.1 Repository layout

Path	Purpose
<code>services/django/lithium</code>	Django project and apps, including ingestion and API logic.
<code>services/frontend/app</code>	React frontend, routes, and UI state.
<code>ops/compose</code>	Docker Compose files for development.
<code>ops/scripts</code>	Helper scripts for environment refreshes.
<code>handover/developer_docs</code>	This documentation.

2.2 Runtime boundaries

- The Django service exposes HTTP endpoints, manages the database, and contains ingestion and matching logic.

- The frontend consumes API endpoints and enforces UI-level access rules for roles.
- Docker Compose wires the services together locally; CI builds and tests both.

2.3 Key data flows (summary)

- Venue upload data creates or updates venue records and their provision capabilities.
- Exam timetable uploads create exams and exam-venue links.
- Provision uploads map student data and allocate students to suitable venues.

Further details are provided in [Section 4](#) and [Section 3](#).

3 Backend overview (Django)

The backend is a Django 5 project located in `services/django/lithium`. It owns the data model, ingestion logic, API endpoints, and admin flows. Most application logic lives in the `timetabling_system` app, with supporting auth in `accounts` and simple page rendering in `pages`.

3.1 Project entry points

- `manage.py`: Django management entry point used by scripts and CI.
- `django_project/settings.py`: configuration, installed apps, middleware, environment defaults.
- `django_project/urls.py`: top-level URL routing (API, auth, pages).
- `django_project/asgi.py` and `django_project/wsgi.py`: deployment entry points.

3.2 Apps and responsibilities

App	Responsibility
<code>timetabling_system</code>	Core domain models, ingestion pipeline, matching logic, API endpoints.
<code>accounts</code>	Authentication, user profiles, invigilator metadata, custom adapters.
<code>pages</code>	Simple HTML pages (marketing/static).

3.3 Settings and environment

The backend reads database settings from environment variables and ships defaults for local development. The Docker service spins up PostgreSQL in-container; in CI, the same environment variables are used. See `README.md` for the current default variable names and test commands.

3.4 Permissions and roles

The system distinguishes admin users from invigilators. Admins can manage all data and upload spreadsheets; invigilators are restricted to their own schedules and availability. API endpoints enforce role-based access; frontend routes apply additional UI guards (see [Section 5](#)).

3.5 Next sections

Detailed model-by-model and function-by-function references are in [Section 7](#) and [Section 9](#). Ingestion details are in [Section 4](#).

4 Ingestion pipeline overview

The ingestion pipeline transforms Excel uploads into normalized domain records. It is the highest-impact logic in the system and has strict ordering requirements: venues first, then exams, then provisions. The primary implementation lives in `timetabling_system/services` and `timetabling_system/utils`.

4.1 Upload sequence

1. **Venue uploads:** create or update `Venue` rows, including capacity, type, accessibility, and availability.
2. **Exam uploads:** create or update `Exam` rows and initial `ExamVenue` links.
3. **Manual capabilities tagging:** admins annotate venues with provision capabilities before provisioning runs.
4. **Provision uploads:** upsert students and their exams, match required capabilities to venues, and attach students to the right `ExamVenue` rows.

This mirrors the behavior documented in `README.md` and should not be reordered without updating tests and UI guidance.

4.2 Key modules

Module	Purpose
<code>services/upload_processor.py</code>	Orchestrates parsing and record upserts for each upload type.
<code>utils/excel_parser.py</code>	Reads Excel files and standardizes raw rows.
<code>utils/file_classifier.py</code>	Determines upload type based on column signatures.
<code>utils/column_mapper.py</code>	Maps various header names to canonical field names.
<code>services/venue_matching.py</code>	Matches provision requirements to suitable venues and prevents conflicts.
<code>services/venue_stats.py</code>	Computes availability/usage statistics for reporting.

4.3 Matching logic (high-level)

The matching logic prioritizes reusing existing `ExamVenue` rows that already satisfy capability requirements. If none exist, it searches for compatible venues based on required provisions, accessibility, availability, and timing. Conflict checks prevent overlapping allocations unless explicitly allowed for the same exam. For detailed behavior and helper routines, see [Section 9](#).

4.4 Error handling and data hygiene

Ingestion attempts to normalize common variants (e.g., provision phrasing and header names). Unknown or malformed rows are typically skipped with logging; the pipeline favors resilience to messy spreadsheets. Tests in `services/django/lithium/tests/django` encode the expected behavior for edge cases.

5 Frontend overview (React)

The frontend lives in `services/frontend/app`. It is a React 19 + Vite application with role-based routes for admins and invigilators. The UI uses Material UI and talks to the Django API via a small fetch wrapper that injects auth tokens.

5.1 Entry points and configuration

- `src/main.tsx`: application bootstrap, mounts the React tree.
- `src/App.tsx`: top-level app wrapper (themes, routing, providers).
- `src/Routes.tsx`: route definitions and role gating.
- `src/vite.config.mts`: Vite build configuration.

5.2 Routes and role gates

Routes are defined in `src/Routes.tsx`. Two guards mediate access:

- `RequireAuth`: enforces authentication before entering protected routes.
- `RequireRole`: restricts protected routes to a specific role (admin or invigilator).

Admin routes are under `/admin/*`, invigilator routes are under `/invigilator/*`. The login page is public, and unknown routes go to a `NotFound` page.

5.3 API access and auth storage

The fetch helper in `src/utils/api.ts` builds a base API URL from `VITE_API_URL` or the browser origin and injects the `Authorization: Token <token>` header. Auth state is stored in `sessionStorage` when available and can fall back to `localStorage` for legacy users. The helper also supports a public fetch mode for unauthenticated calls.

5.4 UI structure

The React tree splits into admin and invigilator layouts, each with nested pages and dialogs. The admin area includes exam, venue, invigilator, and student management screens plus uploads and notifications. Invigilator pages cover dashboards, timetables, availability, restrictions, and profile data.

Detailed component references are provided in [Section 13](#), with links back to relevant backend API endpoints in [Section 8](#).

6 Operations and CI

This section documents how the system is built, started, and tested locally and in CI. The workflows are Docker-first and are designed to keep environments consistent across developers and pipelines.

6.1 Docker Compose dev stack

The dev stack is defined in `ops/compose/docker-compose.dev.yml`. It builds two services:

- **frontend**: runs the React app on port 3000 and points `VITE_API_URL` to the Django service.
- **django**: runs the Django app on port 8000 and starts an embedded PostgreSQL cluster under `services/django/lithium/.postgres-data`.

The Django container runs `scripts/run-dev-with-postgres.sh`, which bootstraps dependencies, migrations, and the dev server.

6.2 Makefile targets

The `Makefile` wraps common workflows. Key targets:

- `make up`: build services, start containers, run smoke tests.
- `make test`: run full frontend and Django test suites.
- `make migrate`: create and apply Django migrations.
- `make reset-django-db`: wipe local embedded Postgres data.

See `Makefile` for full command definitions and environment assumptions.

6.3 Refresh script

`ops/scripts/dev-refresh.sh` performs a full rebuild cycle: tear down containers, rebuild images, start the stack, apply migrations, and execute tests. It is the fastest path to a clean slate when dependencies or migrations drift.

6.4 Continuous integration

CI builds Docker images for both services and runs the frontend (Vitest) and backend (Django test runner) suites. The pipeline also expects deterministic dependency locks and committed migrations.

6.5 Environments and secrets

Production environments must override default credentials (e.g., `DJANGO_SECRET_KEY` and `DJANGO_DB_PASSWORD`). The backend will refuse to start with dev defaults in production mode.

7 Domain model reference

This section summarizes the core Django models. Each entry is a one-line purpose statement with links to relevant workflows. Detailed field-level notes appear in later sections.

7.1 Enum and choice types

Type	Purpose
<code>ProvisionType</code>	Normalized provision labels used by students and matching logic (see Section 4).
<code>ExamVenueProvisionType</code>	Provision capabilities used for venue matching and allocations.
<code>VenueType</code>	Categorical labels for venue classification and reporting.
<code>ExamTypeChoices</code>	Exam modality labels (on-campus vs online).
<code>DietChoices</code>	Predefined diet windows for scheduling (expand as new cycles are added).
<code>SlotChoices</code>	Morning vs evening availability slots.
<code>InvigilatorQualificationChoices</code>	Qualification labels for invigilator capability tracking.
<code>InvigilatorRestrictionType</code>	Restriction labels used when scheduling invigilators.
<code>AccessibilityFeatures</code>	Venue accessibility metadata.

7.2 Core domain tables

Model	Purpose
Diet	Represents an exam diet window with start/end dates and restriction cutoffs.
Exam	Stores exam metadata such as course code, school, and student counts.
Venue	Represents a physical location with capacity, type, and capabilities.
ExamVenue	Links an exam to a venue and captures timing/provision capability context.
Student	Stores student identifiers and names.
StudentExam	Links students to exams and optionally to matched exam venues.
Provisions	Stores student provisions and extra-time notes per exam.

7.3 Notifications and announcements

Model	Purpose
Notification	Tracks system-generated events for admin or invigilator attention.
Announcement	Admin-authored messages displayed in the UI.
UploadLog	Records upload history for traceability and reporting.

7.4 Invigilator management

Model	Purpose
Invigilator	Profile for invigilators, linked to auth users when present.
InvigilatorQualification	Assigns qualifications to invigilators (unique per invigilator).
InvigilatorRestriction	Records diet-specific restrictions per invigilator.
InvigilatorAvailability	Tracks availability by date/slot.
InvigilatorAssignment	Connects invigilators to exam venues with shift timing and role.

7.5 Where to look next

- Model behaviors and signals: see [Section 12](#).
- API serialization of these models: see [Section 8](#).
- Ingestion mapping into these models: see [Section 4](#).

8 API overview

The API is provided by Django REST Framework and is rooted at `/api/`. URL wiring is in `django_project/urls.py` and `timetabling_system/api/urls.py`. The frontend calls these endpoints via `src/utils/api.ts` (see [Section 5](#)).

8.1 URL routing

Top-level routing:

- `/api/` is handled by `timetabling_system/api/urls.py`.
- `/admin/` is the Django admin UI.
- `/accounts/` exposes django-allauth endpoints.

8.2 Authentication endpoints

The API exposes token-based auth and session management:

- `POST /api/auth/token/login/`: username/password login, returns token and user info.
- `GET /api/auth/me/`: current user payload.
- `GET /api/auth/sessions/`: list active sessions.
- `POST /api/auth/sessions/revoke/`: revoke a session.
- `POST /api/auth/sessions/revoke-others/`: revoke all other sessions.
- `POST /api/auth/logout/`: end the current session.

8.3 Admin CRUD endpoints

The following viewsets are admin-only and use standard REST semantics (list, retrieve, create, update, delete):

- `/api/exams/` (`ExamViewSet`)
- `/api/venues/` (`VenueViewSet`)
- `/api/exam-venues/` (`ExamVenueViewSet`)
- `/api/invigilators/` (`InvigilatorViewSet`)
- `/api/invigilator-assignments/` (`InvigilatorAssignmentViewSet`)
- `/api/announcements/` (`AnnouncementViewSet`; read allowed for invigilators)
- `/api/diets/` (`DietViewSet`)

Bulk delete actions exist on exams, venues, and invigilators via `POST /api/<resource>/bulk-delete/`.

8.4 Ingestion and exports

- `POST /api/exams-upload`: upload Excel files, routes to ingestion pipeline.
- `GET /api/provisions/export/`: export provisions CSV (admin-only).
- `POST /api/invigilators/timetables/export/`: export invigilator timetables CSV.

8.5 Notifications and messaging

- `GET/POST /api/notifications/`: list recent notifications or trigger mail merge (admin).
- `GET /api/invigilator/notifications/`: invigilator notification feed.

8.6 Invigilator self-service endpoints

- `GET /api/invigilator/stats/`: invigilator stats summary.
- `GET/PUT /api/invigilator/availability/`: view/update availability for a diet.
- `GET /api/invigilator/assignments/`: list invigilator assignments.
- `POST /api/invigilator-assignments/<id>/pickup/`: pick up a cancelled shift.
- `POST /api/invigilator-assignments/<id>/request-cancel/`: request cancellation.
- `POST /api/invigilator-assignments/<id>/undo-cancel/`: undo cancellation.

8.7 Serializers

Serializers live in `timetabling_system/api/serializers.py`. Highlights:

- `ExamVenueSerializer` and `ExamVenueWriteSerializer`: read vs write shapes for exam venues.
- `ExamSerializer`, `VenueSerializer`, `VenueWriteSerializer`: admin CRUD payloads.
- `InvigilatorAssignmentSerializer`: includes provision details and coverage metadata.
- `InvigilatorSerializer`: nested create/update for qualifications and restrictions.
- `NotificationSerializer`, `AnnouncementSerializer`, `DietSerializer`.

8.8 Permissions and throttling

Most mutation endpoints require admin access. Invigilator-only actions are guarded by `IsInvigilatorOrAdmin`; senior-admin role gates are enforced for admin promotion actions. Admin-only viewsets often disable throttling to allow bulk operations.

9 Services and utilities

This section summarizes the non-view business logic modules that power ingestion, matching, and parsing. These modules are where most data normalization occurs.

9.1 Service layer

Module	Purpose
<code>services/upload_processor.py</code>	Orchestrates ingestion; routes file types to import routines and records <code>UploadLog</code> .
<code>services/venue_matching.py</code>	Capability matching and conflict detection for venue assignments.
<code>services/venue_stats.py</code>	Aggregates student counts per <code>ExamVenue</code> and estimates core exam sizes.

9.2 Ingestion helpers (`upload_processor.py`)

Core helpers:

- `ingest_upload_result`: main entry point; dispatches by file type and returns summary.
- `_import_exam_rows`: upserts exams and core exam venues.
- `_import_provision_rows`: upserts students, provisions, and matching exam venues.
- `_import_venue_days`: ingests venue availability blocks.
- `_required_capabilities`, `_needs_*`: map provision labels into capability constraints.

These routines are the primary business rules for data consistency. Changes here should be accompanied by Django tests in `services/django/lithium/tests/django`.

9.3 Venue matching (`venue_matching.py`)

- `venue_supports_caps`: checks venue capability coverage.
- `venue_has_timing_conflict`: detects schedule overlap, with optional same-exam allowances.
- `venue_is_available`: checks venue availability by date list.

- `attach_placeholders_to_venue`: upgrades placeholder `ExamVenue` rows when a venue becomes capable.

9.4 Utilities

Module	Purpose
<code>utils/excel_parser.py</code>	Reads Excel files, normalizes headers, and dispatches parsing by file type.
<code>utils/file_classifier.py</code>	Detects exam/provision/venue sheets based on column signatures.
<code>utils/column_mapper.py</code>	Normalizes column names and maps equivalents to canonical names.
<code>utils/file_definitions.py</code>	Required canonical columns for exam/provision files.
<code>utils/equivalents.py</code>	Canonical header mapping for messy Excel files.
<code>utils/venue_parser.py</code>	Parses venue availability spreadsheets using <code>openpyxl</code> .
<code>utils/venue_ingest.py</code>	Upserts venue records from parsed availability data.

9.5 Cross-references

- Ingestion flow overview: [Section 4](#).
- API upload endpoint: [Section 8.4](#).
- Model constraints relevant to matching: [Section 7](#).

10 Testing strategy

The codebase uses Django’s test runner for backend logic and Vitest for frontend UI/state behavior. Tests are grouped by subsystem and focus on ingestion correctness, API permissions, and UI workflows.

10.1 Backend tests (Django)

Django tests live under `services/django/lithium/tests/django`. Major areas:

- Upload pipeline coverage: `test_upload_processor*.py`, `test_excel_parser_errors.py`.
- File classification + venue parsing: `test_file_classifier_and_venue_parser.py`.
- API behaviors and permissions: `test_api_views*.py`, `test_api_crud.py`, `test_api_bulk_delete.py`, `test_api_notifications.py`.
- Models and signals: `test_models_repr.py`, `test_models_misc.py`, `test_signals.py`.
- Matching and stats: `test_venue_matching_and_stats.py`, `test_venue_ingest.py`.
- Auth and roles: `test_auth_api.py`, `test_api_views_permissions.py`.

Account-specific tests live under `services/django/lithium/accounts` (see `tests.py`, `tests_ext.py`, `tests_validators.py`).

10.2 Frontend tests (Vitest)

Frontend tests live under `services/frontend/app/tests`. They cover key pages, dialogs, and role gates:

- Admin UI flows: `tests/pages/admin/*.test.tsx` and `tests/components/admin/*.test.tsx`.

- Invigilator UI flows: `tests/pages/invigilator/*.test.tsx` and `tests/components/invigilator/*.test.tsx`
- Role gating: `tests/components/RequireRole.test.tsx`.

10.3 Running tests

Use `make test` to run both suites inside containers. See Section 6 for Make targets and Compose context.

10.4 Coverage

Coverage workflows are provided via `make coverage` and `make coverage-summary` (see Makefile). Backend coverage is reported per-file, and frontend coverage uses Vitest output.

11 Accounts and authentication

The `accounts` app provides a custom user model, session tracking, and login APIs that integrate with the rest of the platform. It also enforces optional login restrictions for resigned invigilators.

11.1 Models

Model	Purpose
<code>CustomUser</code>	Extends <code>AbstractUser</code> with phone, avatar, and senior-admin flag.
<code>UserSession</code>	Per-login token model for multiple concurrent sessions per user.

11.2 Authentication flow

- `ObtainAuthTokenView` issues a DRF token and records a `UserSession` row.
- `UserSessionAuthentication` validates session tokens and updates `last_seen`.
- `CurrentUserView` exposes profile data and supports PATCH updates (username/email/password/phone/avatar).

11.3 Session management endpoints

- GET `/api/auth/sessions/`: list sessions for the current user.
- POST `/api/auth/sessions/revoke/`: revoke a specific session.
- POST `/api/auth/sessions/revoke-others/`: revoke all other sessions.
- POST `/api/auth/logout/`: revoke the current session.

11.4 Login restrictions

`AccountAdapter.is_login_allowed` optionally blocks login for resigned invigilators when `BLOCK_RESIGNED_INVIGILATORS` is enabled in settings.

11.5 Admin integration

The Django admin exposes `CustomUser` with profile fields and a `UserSession` admin for inspecting session activity.

11.6 Password policy

`ComplexityPasswordValidator` enforces a lower/upper/digit/symbol requirement and is referenced in the settings validators list.

12 Signals and implicit behaviors

Some system behaviors are triggered implicitly via Django signals and model hooks. These are easy to overlook when making changes.

12.1 Signal registration

Signals are registered in `timetabling_system/apps.py` via the app `ready()` hook. This ensures the signal handlers in `timetabling_system/signals.py` are loaded when Django starts.

12.2 Venue signals

- `pre_save Venue`: if a venue has `USE_COMPUTER` capability and its type is not computer-friendly, it is coerced to `COMPUTER_CLUSTER`.
- `post_save Venue`: runs `attach_placeholders_to_venue` to upgrade placeholder `ExamVenue` rows when a venue becomes capable.

These signal behaviors tie into the ingestion and matching logic (see [Section 9](#) and [Section 4](#)).

12.3 Model-level hooks

Some models enforce behavior via `clean()` and `save()` overrides (for example, `StudentExam` validates separate-room capacity and calls `full_clean()` on save). See [Section 7](#) for the affected models.

13 Frontend component map

This section maps the main frontend pages and shared components. It is intended as a navigation aid for maintainers rather than exhaustive API documentation.

13.1 Routing layout

Routing is defined in `src/Routes.tsx` and uses two layout shells:

- `AdminLayout`: wraps all `/admin/*` routes.
- `InvigilatorLayout`: wraps all `/invigilator/*` routes.

13.2 Admin pages

Component	Purpose
<code>pages/admin/Dashboard.tsx</code>	Admin landing page and high-level metrics.
<code>pages/admin/Calendar.tsx</code>	Calendar view for exam schedules and allocations.
<code>pages/admin/Exams.tsx</code>	Exam list, filters, bulk actions.
<code>pages/admin/Exam.tsx</code>	Exam detail view and allocations.

<code>pages/admin/Venues.tsx</code>	Venue list and management.
<code>pages/admin/Venue.tsx</code>	Venue detail view and availability.
<code>pages/admin/Invigilators.tsx</code>	Invigilator list and assignment tools.
<code>pages/admin/Invigilator.tsx</code>	Invigilator profile and schedule controls.
<code>pages/admin/Students.tsx</code>	Student/provision management and allocation fixes.
<code>pages/admin/Profile.tsx</code>	Admin profile and auth settings.

13.3 Invigilator pages

Component	Purpose
<code>pages/invigilator/Dashboard.tsx</code>	Invigilator landing page with summary stats.
<code>pages/invigilator/Timetable.tsx</code>	Upcoming timetable and assignments.
<code>pages/invigilator/Shifts.tsx</code>	Shift pickup and cancellation actions.
<code>pages/invigilator/Restrictions.tsx</code>	Availability and restriction entry.
<code>pages/invigilator/Profile.tsx</code>	Invigilator profile management.

13.4 Admin dialogs and shared components

Admin dialogs live in `src/components/admin`. Key components:

- `UploadFile`: Excel ingestion UI.
- `AssignInvigilatorDialog`, `EditInvigilatorDialog`, `InvigilatorAvailabilityModal`: assignment and availability tooling.
- `DietManager`: diet administration UI.
- `NotificationsPanel`, `NotifyDialog`: admin messaging and mail merge.
- `ExportInvigilatorTimetablesDialog`, `ExportProgressDialog`: CSV export flows.

Shared cross-role components live in `src/components` (e.g., `RequireAuth`, `RequireRole`, `Panel`, `PillButton`).

13.5 Cross-references

- API endpoints used by these pages: [Section 8](#).
- Auth/role logic: [Section 11](#).

14 End-to-end data flow

This section provides a step-by-step view of how data moves through the system, from file upload to allocation and UI display. It is intended as a maintainer guide for debugging ingestion issues.

14.1 Upload endpoint

The admin UI uploads Excel files via `POST /api/exams-upload` (see [Section 8.4](#)). This endpoint is implemented in `TimetableUploadView` and calls `parse_excel_file`.

14.2 File classification and parsing

- `parse_excel_file` uses `pandas` to load a workbook and normalize headers.
- `detect_provision_file` and `detect_exam_file` inspect canonical columns.

- `detect_venue_file` detects structural venue sheets and falls back to `parse_venue_file`.

Required columns are validated against `utils/file_definitions.py` before ingestion proceeds.

14.3 Ingestion and persistence

Once parsing succeeds, `ingest_upload_result` dispatches by file type:

- **Exam:** `_import_exam_rows` upserts `Exam` and creates core `ExamVenue` rows.
- **Provisions:** `_import_provision_rows` upserts `Student`, `Provisions`, `StudentExam`, then allocates venues.
- **Venue:** `_import_venue_days` captures availability and updates venues.

Every successful ingestion writes an `UploadLog` row for traceability.

14.4 Allocation and matching

Provision ingestion is where matching rules apply:

- Required capabilities are derived from provision labels (`_required_capabilities`).
- Candidate venues are filtered by type, accessibility, and availability.
- `venue_has_timing_conflict` prevents overlaps.
- `attach_placeholders_to_venue` upgrades placeholder `ExamVenue` rows when a venue becomes capable.

These rules are validated by upload and matching tests (see Section 10).

14.5 API serialization to UI

Backend model data is serialized via DRF serializers and consumed by the frontend. For example:

- `ExamSerializer` and `ExamVenueSerializer` power admin exam screens.
- `InvigilatorAssignmentSerializer` drives invigilator timetable and shift flows.
- `NotificationSerializer` fuels admin and invigilator notifications.

See Section 8 for endpoint coverage and Section 13 for UI mappings.

15 Glossary and module map

This section defines key domain terms and maps the most important modules to their responsibilities.

15.1 Glossary

Term	Definition
Diet	An exam period window (e.g., April/May) with dates and restriction cutoffs.
ExamVenue	Join model connecting an exam to a venue and timing context.
Provision	A student requirement (extra time, separate room, assistive tech).
Placeholder ExamVenue	An <code>ExamVenue</code> with no venue assigned yet.
Capability	A venue feature used to satisfy provisions (e.g., computer, accessible hall).
Invigilator	Staff profile that can be assigned to exam venues and shifts.

Assignment	A scheduled invigilator shift tied to an exam venue.
Restriction	Invigilator availability limitations for a given diet.

15.2 Module map

Module/Path	Responsibility
<code>timetabling_system/models.py</code>	Domain models and constraints (see Section 7).
<code>timetabling_system/api/</code>	REST API endpoints and serializers (see Section 8).
<code>timetabling_system/services/</code>	Ingestion + matching business logic (see Section 9).
<code>timetabling_system/utils/</code>	Excel parsing and header normalization helpers.
<code>accounts/</code>	Custom user model and auth/session endpoints (see Section 11).
<code>services/frontend/app/src</code>	React frontend and role-gated routes (see Section 5).
<code>ops/compose</code>	Docker compose definitions (see Section 6).

15.3 Known gaps

This documentation focuses on the highest-value maintainer knowledge. Remaining gaps to fill:

- Per-class and per-function reference tables for large files (e.g., `upload_processor.py`).
- Detailed permission matrix per endpoint.
- Frontend state/dataflow diagrams for complex admin flows.

16 Upload processor reference

This section provides a function-level reference for `timetabling_system/services/upload_processor.py`. The intent is to highlight responsibilities and how helpers compose.

16.1 Public entrypoint

Function	Purpose
<code>ingest_upload_result</code>	Dispatches parsed results by file type, persists data, writes UploadLog, and returns a summary.

16.2 Parsing and normalization helpers

Function	Purpose
<code>_base_summary</code>	Initializes summary counters for ingestion.
<code>_maybe_to_datetime</code>	Extracts datetime from pandas-like values when present.
<code>_is_missing</code>	Normalizes missing/blank detection including NaN.
<code>_clean_string</code>	Strips and truncates strings for safe storage.
<code>_coerce_date</code>	Coerces values into <code>date</code> objects when possible.
<code>_coerce_datetime</code>	Coerces values into <code>datetime</code> objects when possible.
<code>_time_from_digits</code>	Interprets digit-only strings as HHMM time values.

<code>_coerce_time</code>	Coerces values into <code>time</code> objects and handles Excel fractions.
<code>_coerce_int</code>	Coerces durations and numeric fields into integers.
<code>_ensure_aware</code>	Normalizes timezone awareness based on settings.
<code>_combine_start_datetime</code>	Builds a start datetime from date + time inputs.
<code>_duration_in_minutes</code>	Derives durations from explicit length or end time.

16.3 Exam ingestion

Function	Purpose
<code>_core_exam_timing</code>	Finds core start time and duration from <code>ExamVenue</code> rows.
<code>_import_exam_rows</code>	Upserts <code>Exam</code> records and creates core <code>ExamVenue</code> links.
<code>_build_exam_payload</code>	Normalizes a raw row into exam fields and timing.
<code>_create_exam_venue_links</code>	Ensures venues exist and links them to exams as core venues.
<code>_extract_venue_names</code>	Parses venue names from raw rows, handling online markers.

16.4 Provision ingestion

Function	Purpose
<code>_normalize_provisions</code>	Maps raw provision text into canonical <code>ProvisionType</code> values.
<code>_match_extra_time_token</code>	Handles extra-time token parsing from messy text.
<code>_import_provision_rows</code>	Upserts students, provisions, and matches/allocates venues.
<code>_required_capabilities</code>	Maps provision labels to venue capability requirements.
<code>_needs_accessible_venue</code>	True if provisions require accessibility.
<code>_needs_separate_room</code>	True if provisions require separate rooms.
<code>_needs_computer</code>	True if provisions require computer access.
<code>_exam_requires_computer</code>	Infers computer requirement from exam type.
<code>_extra_time_minutes</code>	Calculates extra time needed from provisions.
<code>_apply_extra_time</code>	Adjusts start time/length to accommodate extra time.
<code>_has_small_extra_time</code>	Heuristic for small extra time allowances.
<code>_allowed_venue_types</code>	Determines compatible venue types for allocations.
<code>_find_matching_exam_venue</code>	Finds existing <code>ExamVenue</code> matching constraints.
<code>_allocate_exam_venue</code>	Creates or repurposes <code>ExamVenue</code> rows to satisfy needs.
<code>_exam_venue_is_reserved</code>	Enforces separate-room exclusivity.

16.5 Venue availability ingestion

Function	Purpose
<code>_import_venue_days</code>	Upserts venues and availability dates from venue uploads.

16.6 Cross-references

- Data-flow overview: Section 14.
- Matching logic: Section 9.
- Tests covering ingestion: Section 10.

17 API endpoint table

This section lists the primary API endpoints, their handlers, and access requirements. It complements the high-level overview in Section 8.

17.1 Authentication

Endpoint	Access	Handler
POST /api/auth/token/login/	Public	ObtainAuthTokenView
GET /api/auth/me/	Authenticated	CurrentUserView
PATCH /api/auth/me/	Authenticated	CurrentUserView
DELETE /api/auth/me/	Admin only	CurrentUserView
GET /api/auth/sessions/	Authenticated	SessionListView
POST /api/auth/sessions/revoke/	Authenticated	SessionRevokeView
POST /api/auth/sessions/revoke-others/	Authenticated	SessionRevokeOthersView
POST /api/auth/logout/	Authenticated	SessionLogoutView

17.2 Admin CRUD viewsets

Endpoint	Access	Handler
/api/exams/	Admin	ExamViewSet
/api/exams/bulk-delete/	Admin	ExamViewSet.bulk_delete
/api/venues/	Admin	VenueViewSet
/api/venues/bulk-delete/	Admin	VenueViewSet.bulk_delete
/api/exam-venues/	Admin	ExamVenueViewSet
/api/invigilators/	Admin	InvigilatorViewSet
/api/invigilators/bulk-delete/	Admin	InvigilatorViewSet.bulk_delete
/api/announcements/	Admin write, invigilator read	AnnouncementViewSet
/api/diets/	Admin	DietViewSet

17.3 Uploads and exports

Endpoint	Access	Handler
----------	--------	---------

POST /api/exams-upload	Admin	TimetableUploadView
GET /api/provisions/export/	Admin	ProvisionExportView
POST /api/invigilators/timetables/export/	Admin	InvigilatorTimetableExportView

17.4 Notifications and messaging

Endpoint	Access	Handler
GET /api/notifications/	Admin	NotificationsView
POST /api/notifications/	Admin	NotificationsView
GET /api/invigilator/notifications/	Authenticated	InvigilatorNotificationsView

17.5 Invigilator self-service

Endpoint	Access	Handler
GET /api/invigilator/stats/	Authenticated	InvigilatorStatsView
GET /api/invigilator/availability/	Authenticated	InvigilatorAvailabilityView
PUT /api/invigilator/availability/	Authenticated	InvigilatorAvailabilityView
GET /api/invigilator/assignments/	Authenticated	InvigilatorAssignmentsView
GET /api/invigilator-assignments/	Authenticated	InvigilatorAssignmentViewSet.list
POST /api/invigilator-assignments/<id>/pickup/	Authenticated	InvigilatorAssignmentViewSet.pickup
POST /api/invigilator-assignments/<id>/request-cancel/	Authenticated	InvigilatorAssignmentViewSet.request_cancel
POST /api/invigilator-assignments/<id>/undo-cancel/	Authenticated	InvigilatorAssignmentViewSet.undo_cancel

17.6 Student/provision management

Endpoint	Access	Handler
GET /api/students/provisions/	Admin	StudentProvisionListView
PATCH /api/students/provisions/	Admin	StudentProvisionListView
DELETE /api/students/provisions/	Admin	StudentProvisionListView

17.7 Cross-references

- Serializer details: Section [8.7](#).
- Auth/session behavior: Section [11](#).
- Frontend consumers: Section [13](#).

18 API views reference

This section provides a high-level reference to `timetabling_system/api/views.py`. It lists key helpers and view classes with their responsibilities.

18.1 Helper functions

Helper	Purpose
<code>log_notification</code>	Writes a <code>Notification</code> row for admin/invigilator events.
<code>_get_request_user</code>	Resolves a request user, including serializer context.
<code>_resolve_invigilator_for_user</code>	Maps a user to an <code>Invigilator</code> profile if present.
<code>_has_time_conflict</code>	Detects time overlaps between assignments.
<code>_format_dt, _csv_for</code>	CSV formatting helpers for exports.
<code>_provision_row</code>	Builds a provision allocation row for the student/provision API.

18.2 Permission helpers

Permission	Purpose
<code>IsInvigilatorOrAdmin</code>	Allows admin users or authenticated invigilators.
<code>IsSeniorAdmin</code>	Restricts access to senior admins.

18.3 Export and upload views

View	Purpose
<code>ProvisionExportView</code>	CSV export of provision allocations (admin-only).
<code>InvigilatorT timetableExportView</code>	CSV export of invigilator timetables (admin-only).
<code>T timetableUploadView</code>	Accepts Excel uploads and triggers ingestion.

18.4 CRUD viewsets

ViewSet	Purpose
<code>ExamViewSet</code>	Admin CRUD for exams with bulk-delete.
<code>VenueViewSet</code>	Admin CRUD for venues with bulk-delete and write serializer.

<code>ExamVenueViewSet</code>	Admin CRUD for exam-venue links (core rows are protected).
<code>InvigilatorViewSet</code>	Admin CRUD for invigilators plus admin promotion actions.
<code>InvigilatorAssignmentViewSet</code>	Admin CRUD and invigilator self-service actions for shifts.
<code>AnnouncementViewSet</code>	Admin CRUD for announcements; read for invigilators.
<code>DietViewSet</code>	Admin CRUD for diets with cleanup on delete.

18.5 Notifications and messaging

View	Purpose
<code>NotificationsView</code>	Admin list + mail-merge delivery for notifications.
<code>InvigilatorNotificationsView</code>	Invigilator-facing notification feed.

18.6 Invigilator self-service

View	Purpose
<code>InvigilatorAvailabilityView</code>	Read/update invigilator availability for a diet.
<code>InvigilatorStatsView</code>	Summary stats for the authenticated invigilator.
<code>InvigilatorAssignmentsView</code>	List assignments for the authenticated invigilator.

18.7 Student/provision management

View	Purpose
<code>StudentProvisionListView</code>	List, update, and delete student provisions/allocation rows.

18.8 Notes

The views module currently contains duplicated definitions for `DietViewSet` and `_provision_row`. If you modify one, ensure both copies remain consistent or refactor to a single source.

19 API serializer reference

This section summarizes `timetabling_system/api/serializers.py` and the purpose of each serializer.

19.1 Exam and venue serializers

Serializer	Purpose
<code>ExamVenueSerializer</code>	Read serializer for <code>ExamVenue</code> ; includes derived fields like venue name and student counts.

<code>ExamVenueWriteSerializer</code>	Write serializer for <code>ExamVenue</code> ; validates separate-room clashes.
<code>ExamSerializer</code>	Exam payload with nested <code>ExamVenue</code> rows.
<code>VenueSerializer</code>	Venue payload with exam links; filters disallowed capabilities.
<code>VenueWriteSerializer</code>	Write serializer for venues; enforces capability cleanup.

19.2 Notification and announcement serializers

Serializer	Purpose
<code>NotificationSerializer</code>	Serializes notifications with triggering user and invigilator metadata.
<code>AnnouncementSerializer</code>	Serializes announcements for dashboards.

19.3 Invigilator serializers

Serializer	Purpose
<code>InvigilatorAssignmentSerializer</code>	Includes exam/venue context, provisions, and cover status.
<code>InvigilatorQualificationSerializer</code>	Writes qualification entries.
<code>InvigilatorRestrictionSerializer</code>	Writes diet-specific restriction entries.
<code>InvigilatorAvailabilitySerializer</code>	Serializes per-day availability.
<code>InvigilatorSerializer</code>	Full invigilator payload with nested qualifications/restrictions.

19.4 Diet serializer

Serializer	Purpose
<code>DietSerializer</code>	CRUD serializer for diet windows (read-only timestamps).

19.5 Cross-references

- Endpoint usage: [Section 17](#).
- Model reference: [Section 7](#).

20 Model detail reference

This section expands on [Section 7](#) with per-class notes and behavioral constraints for `timetabling_system/models`

20.1 Choice types

All choice classes are `models.TextChoices` used for validation and UI selection. Key types:

- `ProvisionType`: normalized provision labels used in ingestion and matching.
- `ExamVenueProvisionType`: capability labels stored on venues and exam venues.

- `VenueType`, `ExamTypeChoices`, `DietChoices`, `SlotChoices`.
- `InvigilatorQualificationChoices`, `InvigilatorRestrictionType`, `AccessibilityFeatures`.

20.2 Core tables

Model	Notes
<code>Exam</code>	Core exam metadata; <code>exam_id</code> is the primary key.
<code>Venue</code>	Primary key is <code>venue_name</code> ; capabilities and availability are stored in arrays.
<code>ExamVenue</code>	Join model between exam and venue; <code>core</code> marks primary allocations.
<code>Student</code>	Primary key is <code>student_id</code> ; minimal identity data.
<code>StudentExam</code>	Unique per (student, exam). On save, validates separate-room exclusivity.
<code>Provisions</code>	Stores normalized provision list, notes, and optional extra-time details.

20.3 Notifications and uploads

Model	Notes
<code>Notification</code>	Stores admin/invigilator messages; ordered newest-first.
<code>Announcement</code>	Admin-authored messages with audience and expiry controls.
<code>UploadLog</code>	Records file ingest history (created/updated counts).

20.4 Invigilator management

Model	Notes
<code>Invigilator</code>	Profile linked to auth user (optional). <code>preferred_name</code> used in display.
<code>InvigilatorQualification</code>	Unique per invigilator/qualification.
<code>InvigilatorRestriction</code>	Diet-scoped restrictions per invigilator.
<code>InvigilatorAvailability</code>	Availability by date and slot; unique on (invigilator, date, slot).
<code>InvigilatorAssignment</code>	Shift record with timing, role, cancellation, and cover tracking.

20.5 Behavioral hooks

- `StudentExam.clean()` enforces separate-room exclusivity per `ExamVenue`.
- `StudentExam.save()` calls `full_clean()` to ensure validation.
- Venue-related behaviors are augmented by signals (see Section 12).

21 Frontend component reference (detailed)

This section lists each frontend page and component with a brief purpose statement.

21.1 Pages

File	Purpose
src/pages/Login.tsx	Login screen and authentication entry.
src/pages/NotFound.tsx	404 page for unknown routes.
src/pages/admin/Dashboard.tsx	Admin dashboard and summary metrics.
src/pages/admin/Calendar.tsx	Calendar schedule visualization and filters.
src/pages/admin/Exams.tsx	Exam list and admin operations.
src/pages/admin/Exam.tsx	Single exam details and allocations.
src/pages/admin/Venues.tsx	Venue list and management tools.
src/pages/admin/Venue.tsx	Venue detail view and availability.
src/pages/admin/Invigilators.tsx	Invigilator list and bulk actions.
src/pages/admin/Invigilator.tsx	Invigilator profile and assignments.
src/pages/admin/Students.tsx	Student/provision management and reallocations.
src/pages/admin/Profile.tsx	Admin profile and auth settings.
src/pages/invigilator/Dashboard.tsx	Invigilator summary dashboard.
src/pages/invigilator/Timetable.tsx	Invigilator timetable and assignments.
src/pages/invigilator/Shifts.tsx	Shift pickup/cancellation workflow.
src/pages/invigilator/Restrictions.tsx	Availability and restriction editor.
src/pages/invigilator/Profile.tsx	Invigilator profile editor.

21.2 Shared components

File	Purpose
src/components/RequireAuth.tsx	Auth gate for protected routes.
src/components/RequireRole.tsx	Role-based gate for admin/invigilator routes.
src/components/Panel.tsx	Shared panel layout and styling.
src/components/PillButton.tsx	Reusable pill-style button.
src/components/BooleanCheckboxRow.tsx	Boolean checkbox row helper.
src/components/CollapsibleSection.tsx	Expand/collapse container for dense forms.

21.3 Admin components

File	Purpose
src/components/admin/Layout.tsx	Admin layout shell and nav.
src/components/admin/UploadFile.tsx	Upload widget for Excel ingestion.
src/components/admin/AssignInvigilatorDialog.tsx	Assign invigilators to shifts.
src/components/admin/AddInvigilatorDialog.tsx	Create invigilator entries.
src/components/admin/EditInvigilatorDialog.tsx	Edit invigilator profiles and restrictions.
src/components/admin/InvigilatorAvailabilityModal.tsx	Invigilator availability.
src/components/admin/ContractedHoursReport.tsx	Report on contracted hours.
src/components/admin/ExportInvigilatorTimetableDialog.tsx	Export invigilator timetable CSVs.
src/components/admin/ExportProgressDialog.tsx	Export progress indicator.

<code>src/components/admin/EditExamDialog.tsx</code>	Edit exam metadata.
<code>src/components/admin/AddExamDialog.tsx</code>	Create exams.
<code>src/components/admin/ExamDetailsPopup.tsx</code>	Exam details quick view.
<code>src/components/admin/EditVenueDialog.tsx</code>	Edit venue metadata and capabilities.
<code>src/components/admin/AddVenueDialog.tsx</code>	Create venues.
<code>src/components/admin/DeleteConfirmationDialog.tsx</code>	Confirmation for destructive actions.
<code>src/components/admin/DietManager.tsx</code>	Create/edit diets.
<code>src/components/admin/NotificationsPanel.tsx</code>	Admin notifications feed.
<code>src/components/admin/NotifyDialog.tsx</code>	Mail merge/message tool.
<code>src/components/admin/AddAnnouncementDialog.tsx</code>	Create announcements.

21.4 Invigilator components

File	Purpose
<code>src/components/invigilator/Layout.tsx</code>	Invigilator layout shell and nav.
<code>src/components/invigilator/ShiftPickupDialog.tsx</code>	Shift pickup action UI.

21.5 Cross-references

- Routes and role gates: Section 5.
- API usage: Section 17.

22 Ops scripts and container images

This section documents the operational scripts and Dockerfiles used to run the system locally and in CI.

22.1 Docker Compose (dev)

`ops/compose/docker-compose.dev.yml` defines the two service stack. The frontend binds port 3000, and Django binds port 8000. The Django service runs `scripts/run-dev-with-postgres.sh` as its command.

22.2 Django Dockerfile

`services/django/lithium/Dockerfile` is a two-stage build:

- Builder uses `uv` to sync dependencies into `/app`.
- Final image installs system build tooling and PostgreSQL 15 packages.
- Default CMD runs Gunicorn on port 8000.

22.3 Frontend Dockerfile

`services/frontend/Dockerfile` builds a Node 20 slim image, installs build tooling, copies the app skeleton, and runs a small entrypoint script. The container defaults to an idle command until overridden by Compose.

22.4 Django runtime scripts

- `scripts/run-dev-with-postgres.sh`: boots embedded Postgres, prepares venv, runs migrations, and starts `runserver`.
- `scripts/manage.sh`: local helper to create/reuse a venv and run management commands.
- `scripts/create-timetable-app.sh`: bootstrap utility for creating the app scaffold inside containers.

22.5 Frontend runtime scripts

- `app/bin/start.sh`: installs dependencies if missing and runs the Vite dev server.
- `app/bin/test.sh`: installs deps if needed and runs the npm test script.

22.6 Cross-references

- Dev workflows: [Section 6](#).
- Make targets: [Section 6.2](#).

23 Accounts app reference

This section provides a function/class reference for the `accounts` app.

23.1 Models

Model	Purpose
<code>CustomUser</code>	Extends <code>AbstractUser</code> with phone, avatar, and senior-admin flag.
<code>UserSession</code>	Per-login token model; supports multiple concurrent sessions per user.

23.2 Auth API helpers and views

Helper/View	Purpose
<code>_safe_attr</code>	Safe attribute access to avoid exploding on missing attributes.
<code>_derive_role</code>	Derives role label from user flags/profile.
<code>_get_invigilator_id</code>	Pulls invigilator id from the user profile.
<code>_get_senior_admin_flag</code>	Checks the senior-admin flag.
<code>_get_client_ip</code> , <code>_get_user_agent</code>	Capture request metadata for session tracking.
<code>AuthTokenSerializer</code>	Validates username/password (username or email).
<code>ObtainAuthTokenView</code>	Issues DRF token and logs a <code>UserSession</code> .
<code>CurrentUserView</code>	GET/PATCH/DELETE for profile updates and admin self-delete.
<code>SessionListView</code>	Lists sessions for the current user.
<code>SessionRevokeView</code>	Revokes a specific session.

<code>SessionRevokeOthersView</code>	Revokes all other active sessions.
<code>SessionLogoutView</code>	Revokes the current session.

23.3 Authentication backend

Class	Purpose
<code>UserSessionAuthentication</code>	DRF auth class that validates <code>UserSession</code> tokens and updates <code>last_seen</code> .

23.4 Adapters and validators

Class	Purpose
<code>AccountAdapter</code>	Blocks resigned invigilator login when configured.
<code>ComplexityPasswordValidator</code>	Enforces lower/upper/digit/symbol complexity.

23.5 Admin integration

Class	Purpose
<code>CustomUserAdmin</code>	Admin UI for custom users with profile fields.
<code>UserSessionAdmin</code>	Admin UI for viewing and revoking sessions.

24 Backend utilities reference

This section summarizes per-function responsibilities for the key helper modules under `timetabling_system/utills` and `timetabling_system/services` not covered elsewhere.

24.1 Venue matching and stats

Function	Purpose
<code>venue_supports_caps</code>	Validates that a venue has all required provision capabilities.
<code>venue_has_timing_conflict</code>	Detects overlapping <code>ExamVenue</code> slots for a venue.
<code>venue_is_available</code>	Checks a venue's availability list for a date.
<code>attach_placeholders_to_venue</code>	Binds placeholder <code>ExamVenue</code> rows to a newly capable venue.
<code>examvenue_student_counts</code>	Aggregates student counts per <code>ExamVenue</code> .
<code>core_exam_size</code>	Estimates core exam size excluding extra-time allocations.

24.2 Excel parsing and classification

Function	Purpose
<code>parse_excel_file</code>	Main Excel parser; classifies and returns structured rows.
<code>prepare_exam_provision_df</code>	Normalizes and cleans DataFrames for exam/provision parsing.
<code>validate_required_columns</code>	Checks column requirements from <code>file_definitions</code> .
<code>detect_provision_file</code>	Detects provision files by column signatures.
<code>detect_exam_file</code>	Detects exam files by column signatures.
<code>detect_venue_file</code>	Detects structural venue sheets.

24.3 Column normalization

Function	Purpose
<code>normalize</code>	Normalizes column labels to lowercase, underscore-separated tokens.
<code>map_equivalent_columns</code>	Maps messy headers to canonical names.

24.4 Venue parsing and ingest

Function	Purpose
<code>parse_venue_file</code>	Parses venue availability spreadsheets using <code>openpyxl</code> .
<code>_cell_to_date_text</code>	Converts Excel date cells into ISO strings.
<code>upsert_venues</code>	Upserts venues from parsed venue files with defaults.

24.5 Cross-references

- Ingestion pipeline: [Section 4](#).
- Upload processor reference: [Section 16](#).

25 Frontend state and data flow

This section summarizes how the frontend retrieves data, stores auth state, and coordinates admin workflows. It does not document every hook or state variable, but highlights the most important flows.

25.1 Auth state

- Auth tokens and user payloads are stored via `src/utils/api.ts` using `sessionStorage` when available (with legacy migration from `localStorage`).
- `RequireAuth` and `RequireRole` gate route access.
- Logout clears stored auth keys and returns to `/login`.

25.2 Admin data flow (common patterns)

Admin pages generally follow a request-refresh pattern:

- Fetch list data from `/api/*` endpoints on mount.
- Use modal dialogs (add/edit/delete) to submit mutations.
- On success, re-fetch the list or update local state.

Key workflows:

- **Uploads:** `UploadFile` posts to `/api/exams-upload` and displays ingestion summaries.
- **Invigilator assignments:** dialogs post to assignment endpoints and refresh the invigilator list.
- **Student provisions:** `AdminStudents` calls `/api/students/provisions` and updates allocations with `PATCH/DELETE`.
- **Exports:** export dialogs call CSV endpoints and show progress feedback.

25.3 Invigilator data flow

Invigilator pages focus on a single user profile:

- Timetable and shifts load assignments from `/api/invigilator-assignments`.
- Availability reads and writes `/api/invigilator/availability`.
- Notifications poll `/api/invigilator/notifications`.

25.4 Cross-references

- API endpoints: [Section 17](#).
- Auth details: [Section 11](#).

26 Accounts tests and migrations

This section describes the accounts-specific tests and migrations that shape authentication behavior.

26.1 Default admin migration

`accounts/migrations/0002_create_default_admin.py` seeds a default admin user and token for local use. It provides:

- `create_default_admin`: creates or updates the default admin user and token.
- `remove_default_admin`: removes the default admin user on reverse migration.

These behaviors are covered by migration tests in `accounts/tests_ext.py`.

26.2 Core account tests

`accounts/tests.py` covers basic user creation, login page rendering, and admin login redirects.

26.3 Extended auth tests

`accounts/tests_ext.py` covers:

- Account adapter behavior for resigned invigilators.
- Auth API edge cases (missing credentials, inactive users).
- Current user `PATCH` flows (username/email/password/avatar/phone).
- Session token regeneration and deletion behavior.

- Migration logic for default admin creation/removal.

26.4 Password validator tests

`accounts/tests_validators.py` verifies the complexity validator accepts valid passwords and rejects weak ones.

27 Backend test index

This section lists Django test files and their primary focus. Use this as a guide when modifying behavior.

Test file	Coverage focus
<code>test_announcements_api.py</code>	Announcement API behaviors.
<code>test_api_bulk_delete.py</code>	Bulk delete endpoints for admin CRUD.
<code>test_api_crud.py</code>	Core CRUD behaviors for major models.
<code>test_api_notifications.py</code>	Notification API and mail merge behaviors.
<code>test_api_serializers_views.py</code>	Serializer and view integration checks.
<code>test_api_views.py</code>	General API view behavior.
<code>test_api_views_permissions.py</code>	Role/permission enforcement in views.
<code>test_auth_api.py</code>	Auth API edge cases and login behavior.
<code>test_diets_api.py</code>	Diet CRUD and availability logic.
<code>test_excel_parser_errors.py</code>	Parser error handling for invalid files.
<code>test_file_classifier_and_venue_parser.py</code>	File classification + venue parsing.
<code>test_invigilator_stats.py</code>	Invigilator stats endpoint logic.
<code>test_models_misc.py</code>	Misc model behaviors and enums.
<code>test_models_repr.py</code>	Model <code>__str__</code> representations.
<code>test_settings_helpers.py</code>	Settings utilities and toggles.
<code>test_signals.py</code>	Signal behaviors for venues.
<code>test_upload_processor.py</code>	End-to-end ingestion from parsed rows.
<code>test_upload_processor_branches.py</code>	Branch coverage for ingestion paths.
<code>test_upload_processor_edges.py</code>	Edge cases in ingestion/matching.
<code>test_upload_processor_utils.py</code>	Helper functions in upload processor.
<code>test_upload_view.py</code>	Upload endpoint behavior.
<code>test_utils.py</code>	Misc utility helpers.
<code>test_venue_ingest.py</code>	Venue upsert behaviors.
<code>test_venue_matching_and_stats.py</code>	Matching and stats computations.

28 Frontend tests and config

This section lists frontend test files and notes test configuration.

28.1 Vitest configuration

- Test setup lives in `app/setupTests.ts`.
- Vite/Vitest config is in `app/vite.config.mts` and `app/vitest.config.mts`.

28.2 Frontend test index

Test file	Coverage focus
components/RequireRole.test.tsx	Role gating behavior.
components/admin/AddInvigilatorDialog.diets.test.tsx	Invigilator creation with diets.
components/admin/AssignInvigilatorDialog.test.tsx	Assigning invigilators to shifts.
components/admin/ContractedHoursReport.test.tsx	Contracted hours report rendering.
components/admin/DietManager.test.tsx	Diet CRUD UI logic.
components/admin/EditInvigilatorDialog.diets.test.tsx	Invigilator edit flow with diets.
components/admin/ExamDetailsPopup.test.tsx	Exam details popup behavior.
components/admin/ExportInvigilatorTimetablesDialog.test.tsx	Export dialog behavior.
components/admin/InvigilatorAvailabilityModal.test.tsx	Availability modal behavior.
components/admin/UploadFile.test.tsx	Upload UI and error handling.
components/admin/layout.test.tsx	Admin layout rendering.
components/invigilator/layout.test.tsx	Invigilator layout rendering.
components/invigilator/shift-pickup-dialog.test.tsx	Shift pickup dialog flow.
pages/admin/announcements.test.tsx	Admin announcements page.
pages/admin/calendar.test.tsx	Admin calendar UI.
pages/admin/dashboard-export.test.tsx	Dashboard export workflows.
pages/admin/exam-details.test.tsx	Admin exam detail page.
pages/admin/invigilator-profile.test.tsx	Invigilator profile page.
pages/admin/invigilators-export.test.tsx	Invigilator export workflow.
pages/admin/profile.test.tsx	Admin profile page.
pages/admin/statePersistence.test.tsx	State persistence across sessions.
pages/invigilator/announcements.test.tsx	Invigilator announcements page.
pages/invigilator/dashboard.test.tsx	Invigilator dashboard.
pages/invigilator/restrictions.test.tsx	Restrictions/availability page.
pages/invigilator/shifts.test.tsx	Invigilator shifts page.

29 Django views, URLs, and throttling

This section documents the remaining Django views and routing that are not part of the REST API.

29.1 timetabling_system views

View/Function	Purpose
HomePageView	Static landing page for the timetabling system.
AboutPageView	Static about page for the timetabling system.
healthz_view	Health check endpoint that verifies DB connectivity.
upload_timetable_file	Form-based upload endpoint for authenticated users (legacy).

29.2 pages app views

View/Function	Purpose
<code>pages.HomePageView</code>	Static home page for the pages app.
<code>pages.AboutPageView</code>	Static about page for the pages app.
<code>pages.healthz_view</code>	Health check endpoint (pages app).

29.3 URL routing

- `timetabling_system/urls.py` wires `home/about/healthz/upload` views.
- `pages/urls.py` wires pages app `home/about/healthz` views.
- `django_project/urls.py` mounts both and the REST API.

29.4 Throttling

Class	Purpose
<code>AdminBypassUserRateThrottle</code>	Skips user throttling for staff/superusers.

29.5 Cross-references

- API endpoints: [Section 17](#).
- Health endpoint: [Section 6](#).

30 Frontend utilities and state

This section documents utility modules and state setup used across the frontend.

30.1 Utilities

Module	Purpose
<code>src/utils/api.ts</code>	API base URL resolution, auth token storage, and fetch helpers.
<code>src/utils/dates.ts</code>	Date/time formatting helpers for UI.
<code>src/utils/globalStyles.tsx</code>	Shared global styling configuration.
<code>src/theme.ts</code>	MUI theme definitions and overrides.

30.2 State management

- `src/state/store.ts` defines the central store (Redux or context-based, depending on current implementation).
- Page-level state is typically local to the component and refreshed after mutations.

30.3 Cross-references

- Auth flow: [Section 25](#).
- API endpoints: [Section 17](#).